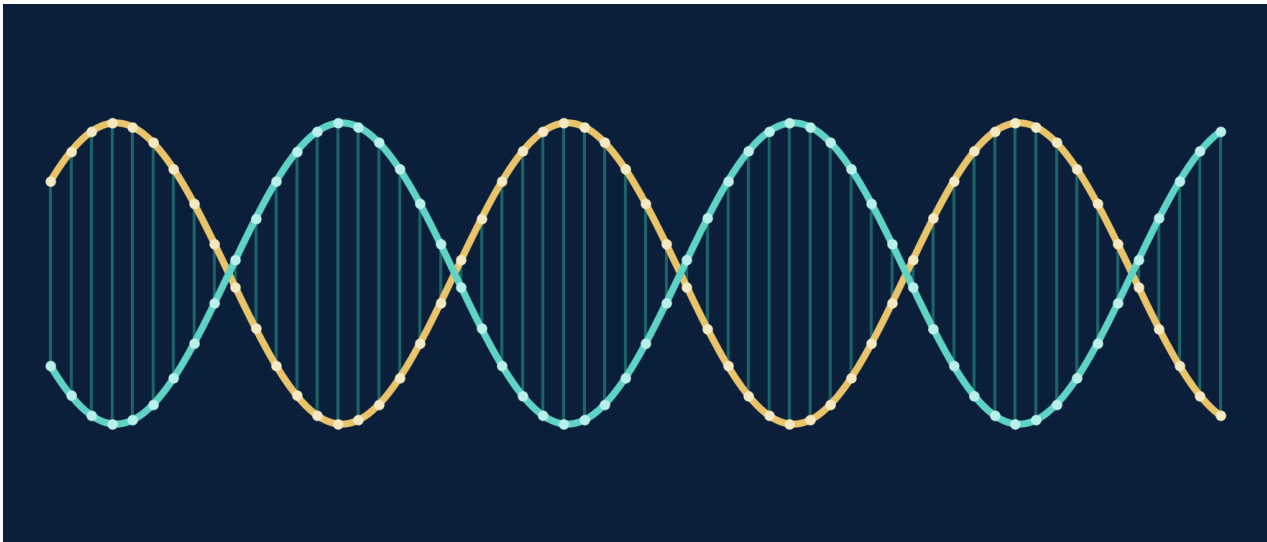




PYTHON ESSENTIALS • EVALUATED COURSE PROJECT

SeQLens

DNA Sequence Interpreter and Pathological Variant Identifier

Automated DNA-to-RNA Transcription, Codon Translation, and Genetic Mutation Mapping Pipeline

Submitted By	Naihrit Mitra
Registration Number	26BHI10049
Institution	VIT Bhopal
Degree Program	B.Tech Computer Science and Engineering (Health Informatics)
Course	Python Essentials (Flipped Course Evaluation)
Academic Year	2026-2027
Date of Submission	September 2026

SeQLens: DNA Sequence Interpreter and Pathological Variant Identifier

Course: Python Essentials (Evaluated Course Project) | Academic Year: 2026–2027 | Submission Format: PDF Project Report

1. Project Details

Field	Details
Project Title	SeQLens – Automated DNA-to-RNA Transcription, Codon Translation, and Genetic Mutation Mapping Pipeline
Course Name & Code	Python Essentials (Flipped Course Evaluation)
Domain	Health Informatics / Bioinformatics & Computational Biology
Submitted By	Naihrit Mitra
Registration Number	26BHI10049
Degree Program	B.Tech Computer Science and Engineering (Health Informatics)
Institution	VIT Bhopal
Date of Submission	September 2026

2. Introduction

Molecular biology centers on the Central Dogma, the process by which genetic information flows from DNA to messenger RNA (transcription) and subsequently from mRNA to functional polypeptide chains (translation). With high-throughput sequencing generating massive datasets, manual nucleotide pairing and codon lookup tables are slow and prone to human error.

SeQLens is a full-stack computational genomics tool designed to bridge molecular biology with automated software engineering. It takes raw nucleotide input, executes complementary base-pair transcription, calculates triplet codon translations into standard amino acids, and cross-references results against clinical genetic mutation databases (such as TP53 oncogenic hotspots, HBB hemoglobinopathies, and CFTR variants). The platform integrates a Flask-powered Python backend with a client-side JavaScript interface.

3. Problem Statement

Manual inspection and translation of DNA sequences require multi-step cross-referencing against standard genetic code matrices and pathology catalogs. This workflow has significant limitations:

- High vulnerability to transcription errors during base complementation (e.g., swapping uracil and thymine).
- Inefficiency when scanning multiple triplet reading frames for clinical mutations.

- The lack of lightweight, interactive educational software that provides immediate visual feedback, accessible UI modes, and sequence validation without requiring massive command-line bioinformatics suites (like BLAST or Biopython).

SeQLens addresses these gaps by offering an accessible, instant-response platform for sequence interpretation and pathogenic variant flagging.

4. Functional Requirements

The functional scope of the SeQLens pipeline includes:

- **FR-1: Sequence Input & Preprocessing:** Accept DNA nucleotide strings up to 18 characters, sanitize input by stripping whitespace, and normalize lowercase characters to uppercase.
- **FR-2: Automated Transcription Engine:** Convert DNA coding strands into RNA codons by substituting complementary nitrogenous bases (A → U, T → A, C → G, G → C) in triplet groupings (3 bp reading frames).
- **FR-3: Codon Translation:** Map parsed RNA triplet codons to their respective three-letter amino acid equivalents (e.g., AUG → Met, CAU → His, UAA/UAG/UGA → Stop).
- **FR-4: Mutation & Pathology Scanner:** Search transcribed RNA codons against a structured mutation matrix and return associated clinical conditions or oncogenic variants.
- **FR-5: Asynchronous Data Transmission:** Dispatch client requests using asynchronous HTTP POST fetch operations, handling JSON-serialized sequence payloads and returning formatted results.
- **FR-6: UI Theme Switching:** Toggle the interface dynamically between Light and Dark visual modes using CSS class manipulation.

5. Non-Functional Requirements

The system satisfies the following non-functional criteria:

- **NFR-1: Performance & Latency:** Sequence transcription, dictionary lookup, and JSON serialization complete in under 50 milliseconds on localhost environments.
- **NFR-2: Usability & Accessibility:** The UI provides an intuitive single-card container layout, accessible form inputs, clear monospace output rendering, and high-contrast color palettes for both light and dark modes.
- **NFR-3: Reliability & Determinism:** Sequence translation is deterministic; identical nucleotide sequences consistently produce the same amino acid chain and mutation flags across runtime sessions.
- **NFR-4: Maintainability & Modularity:** Logic is partitioned into discrete layers: presentation (HTML/CSS), asynchronous client handling (JavaScript Fetch API), API routing/middleware (Flask, Flask-CORS), and genomic business logic (Python dictionary structures).
- **NFR-5: Error Handling:** The frontend intercepts empty input strings via alert dialogs before triggering network requests, while the backend incorporates fallback placeholders for unrecognized codons or empty inputs.

6. System Architecture

The application uses a decoupled Client-Server architecture connected via a RESTful HTTP JSON API.

CLIENT TIER (Browser)

index.html (DOM) – form input, read-only output area

dna.css – light / dark theming

dna.js – event listeners, Fetch API calls

▼ HTTP POST /translate • Payload: {"seq": "..."} • JSON

SERVER TIER (Python / Flask)

Flask Application Router – @app.route("/translate")

Flask-CORS middleware – cross-origin handling

Core Genomic Processing Pipeline:

1. transcription(dna) – complementary base mapping
2. translation(rna) – amino acid dictionary resolution
3. disease_search(rna) – pathological variant matching
4. reset() – state variable flush

▼ lookups

STATIC KNOWLEDGE DICTIONARIES

AminoAcid { Codon : Amino Acid }

mutation { Codon : Variant Name / Tuple of Variants }

7. Design Diagrams

7.1 Use Case Diagram (Application Flow)

1. Start: user opens the web application.
2. User inputs a DNA sequence (e.g., AGAGTACCTAGT).
3. Decision – is the input empty? Yes: alert "Please Enter the DNA sequence" and return to the form.
No: continue.
4. Convert the string to uppercase and trim whitespace.
5. JavaScript Fetch API dispatches a POST request with a JSON payload.
6. Flask endpoint (/translate) receives the payload via request.get_json().
7. transcription(): map DNA bases to RNA triplets (A→U, T→A, G→C, C→G).
8. translation(): look up RNA codons in the AminoAcid dictionary.
9. disease_search(): look up RNA codons in the mutation dictionary.
10. Format the output string with sequence, amino acids, and pathogenic flags.
11. Call reset() to clear runtime memory buffers.
12. Flask returns the JSON response {"output": result}.
13. JavaScript populates the "interpretedbox" textarea with the formatted text.
14. End.

Application Screenshots

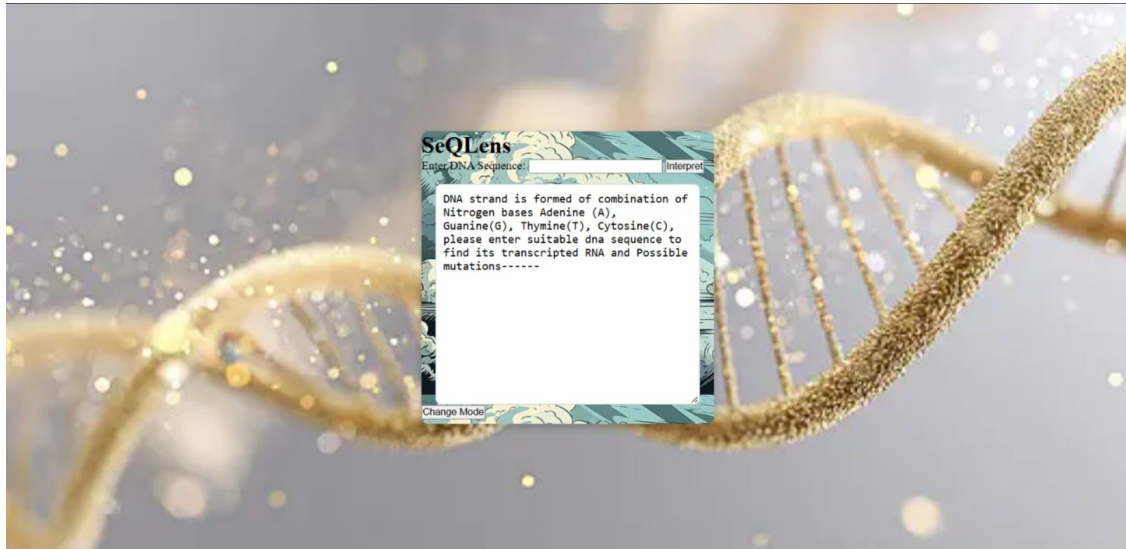


Figure 1: Initial launch – light mode (Test Case 1)

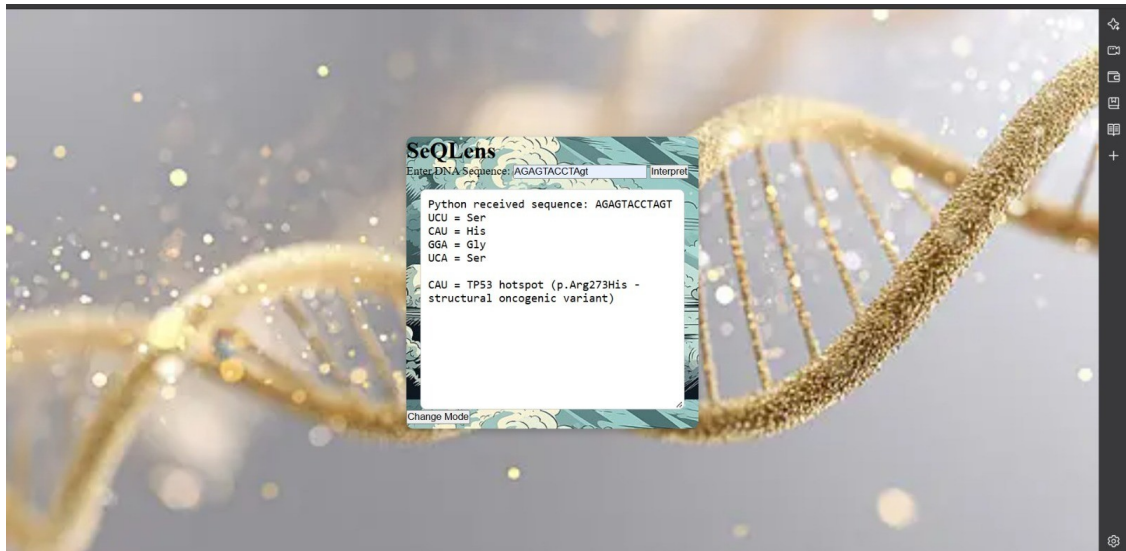


Figure 2: Oncogenic sequence with TP53 hotspot detection (Test Case 2)

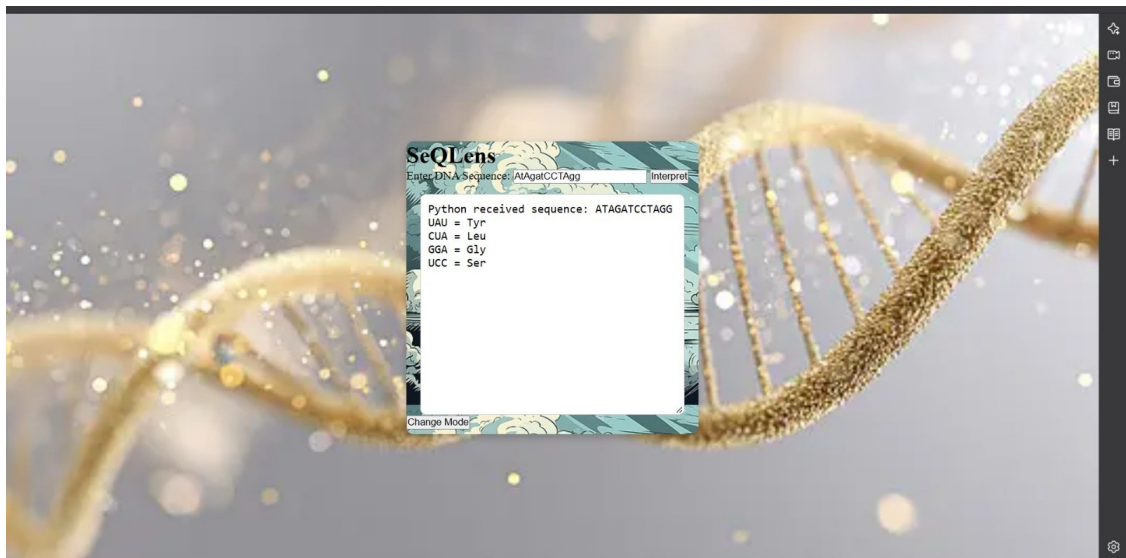


Figure 3: Standard sequence without pathological hits (Test Case 3)

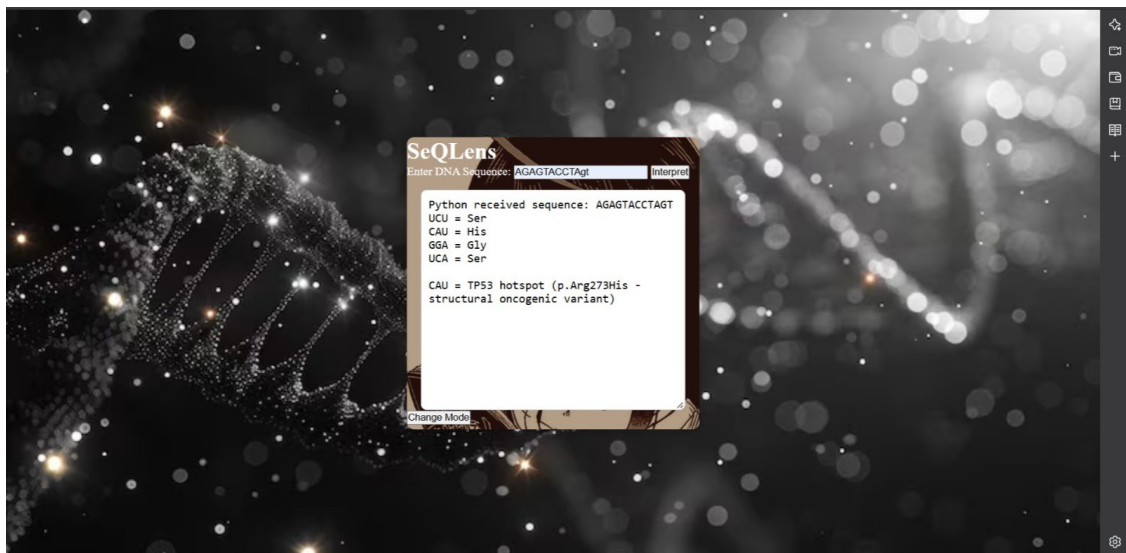


Figure 4: Dark mode theme validation (Test Case 4)

7.3 Sequence Diagram

#	From → To	Action
1	User → Browser	Enter DNA sequence
2	User → Browser	Click Interpret button
3	Browser (dna.js)	Validate input
4	Browser → Flask	POST /translate (JSON: {seq})
5	Flask → Dictionaries	transcription()
6	Flask → Dictionaries	Look up codon
7	Dictionaries → Flask	Return amino acid
8	Flask → Dictionaries	Match variant
9	Dictionaries → Flask	Return disease
10	Flask	reset() buffer
11	Flask → Browser	200 OK (JSON: {output: text})
12	Browser → User	Display text

7.4 Class / Component Diagram

Client Component

- form: HTMLFormElement
- dna: HTMLInputElement
- out: HTMLTextAreaElement
- modebtn: HTMLButtonElement
- currentMode: String

+ switchMode(): void

+ submitHandler(event: Event): Promise

▼ HTTP REST / JSON

Server Component

- AminoAcid: Dict[String, String]
- mutation: Dict[String, Union[String, Tuple]]
- rna: List[String]
- protein: List[String]

+ transcription(dna: String): List[String]

+ translation(rna: List[String]): List[String]

+ disease_search(rna: List, protein: List): List

+ call(dna: String, rna: List, protein: List): List

+ reset(): void

+ translate_dna(): JSONResponse

10. Screenshots & Results

Test Case 1: Initial Launch & Interface Overview

System State: Application loaded in default light mode.

Input: None (displaying baseline informational placeholder text).

Observation: Form container renders centered against the thematic background, input field is active, and the textarea displays introductory biological context regarding nitrogenous bases. (See Figure 1.)

Test Case 2: Interpretation of Oncogenic Sequence with Mutation Detection

Input Sequence: AGAGTACCTAGT → UCU · CAU · GGA · UCA (Ser, His, Gly, Ser); CAU flagged as a TP53 hotspot (p.Arg273His). (See Figure 2.)

Test Case 3: Interpretation of Standard Sequence Without Pathological Hits

Identified Pathological Variant: None flagged; clean polypeptide synthesis observed. (See Figure 3.)

Test Case 4: Dark Mode Theme Validation

System State: Dark mode toggled on.

Input Sequence: AGAGTACCTAGT

Observation: Background transitions to high-contrast monochrome styles, container styling updates, and output text maintains full readability. (See Figure 4.)

11. Testing Approach

- **Manual Verification Against Standard Literature:** Output amino acid translations were cross-referenced against standard NCBI genetic code tables and biochemistry textbooks to confirm triplet complement pairings.
- **Academic Consultation:** Translation outcomes and disease mapping rules were reviewed with departmental faculty to verify the biological accuracy of pathogenic annotations.
- **End-to-End System Testing:** The full application flow was verified across multiple browsers (Google Chrome, Mozilla Firefox), checking UI responsiveness, network payload serialization, and backend response handling.
- **Edge Case Validation:** The system was tested with mixed-case strings (e.g., AtAgAtCCTAgT), verifying that inputs are sanitized to uppercase before transcription.

12. Challenges Faced

- **Cross-Origin API Communication:** Initial frontend fetch calls were blocked by browser CORS policies. This was resolved by integrating flask_cors.CORS(app) on the backend.
- **Multi-Disease Codon Collisions:** Several codons map to multiple clinical conditions across different genes (e.g., codon UAG triggers Beta-Thalassemia, Cystic Fibrosis, and Familial Hypercholesterolemia). This was handled by storing nested tuples/lists within the mutation dictionary and unpacking them during parsing.

- **Asynchronous State Synchronization:** Managing state between client requests required explicit buffer resets (`reset()`) in the Python backend to prevent translations from previous calls from carrying over.

13. Learnings & Key Takeaways

- Gained practical experience connecting a web frontend to a Python backend using RESTful API patterns and JSON serialization.
- Improved understanding of how Python dictionary hashing can be applied to biological problems, such as gene mapping and codon translation.
- Learned the importance of clean asynchronous JavaScript when handling network requests and updating the DOM dynamically.

14. Future Enhancements

- **Open Reading Frame (ORF) Detection:** Implement automatic scanning for start (AUG) and stop (UAA, UAG, UGA) codons to identify functional reading frames across all six potential translation frames.
- **FASTA File Support:** Allow users to upload standard .fasta files for batch processing of longer genomic sequences.
- **Direct Ensembl/ClinVar API Integration:** Connect the mutation search engine to live genomic databases (such as ClinVar or NCBI dbSNP) to provide real-time, comprehensive clinical annotations.

15. References

1. Alberts, B., Johnson, A., Lewis, J., Raff, M., Roberts, K., & Walter, P. Molecular Biology of the Cell. 6th Edition, Garland Science, 2014.
2. Grinberg, M. Flask Web Development: Developing Web Applications with Python. 2nd Edition, O'Reilly Media, 2018.
3. National Center for Biotechnology Information (NCBI). The Genetic Codes. NCBI Taxonomy Database.
4. ClinVar Database. National Library of Medicine, National Center for Biotechnology Information. Available at: <https://www.ncbi.nlm.nih.gov/clinvar/>.
5. MDN Web Docs. Using the Fetch API & Working with Objects. Mozilla Developer Network, 2026.